

Trabalho A2 — Grupo 07: Telemetria IoT

Universidade Estadual do Tocantins — UNITINS Curso: Bacharelado em Sistemas de Informação

Disciplina: Estrutura de Dados — 2026.1 **Professor:** Alysson Martins Bruno **Entrega:** 08/06/2026, até 23h59 (via Educ@) **Valor:** 8,0 pontos

1 Apresentação

A **Avaliação A2** consiste na implementação, em Java, de um **SGBD didático do tipo chave-valor**, com persistência em disco e uso de uma estrutura de dados estudada na disciplina.

Este enunciado é específico do **Grupo 07 — Telemetria IoT**. Todos os grupos compartilham um **núcleo comum** e variam apenas no contexto e na estrutura adicional pedida.

O trabalho foi dimensionado para:

- grupos de **2 ou 3 estudantes**;
 - cerca de **3 horas de trabalho efetivo**;
 - foco em **núcleo funcional**, não em um sistema completo de produção.
-

2 Contexto do Grupo

Domínio: leituras numéricas de um sensor IoT. **Chave do banco:** timestamp da leitura (**String** representando um número inteiro, por exemplo "100", "205", "330"). **Valor do banco:** valor numérico lido pelo sensor, gravado como **String** (por exemplo, "23.5").

Observação sobre os tipos: o contrato público é `put(String, String)`. O grupo pode adicionar um método auxiliar `gravarLeitura(int timestamp, double valor)` na implementação concreta, que internamente chama `put(String.valueOf(timestamp), String.valueOf(valor))`.

3 Núcleo Comum Obrigatório

3.1 Operações básicas

```
public interface SGBD {  
    void put(String chave, String valor);  
    String get(String chave);  
    void delete(String chave);  
    void fechar();  
}
```

3.2 Persistência obrigatória

- append-only log;

- registro de cada `put` e `delete`;
- reconstrução ao iniciar;
- `flush + force(true) / fsync`.

3.3 Estrutura principal em memória

- `HashMap<String, String>` para acesso direto por timestamp;
- **BST** indexada por timestamp como estrutura secundária (ver seção 4).

3.4 Organização mínima sugerida

```
src/  
  SGBD.java  
  TipoOperacao.java  
  RegistroLog.java  
  LogAppendOnly.java  
  Telemetria.java           (implementação concreta)  
  NoBST.java               (estrutura específica do grupo)  
  DemoGrupo07.java
```

3.5 Entrega mínima

1. código-fonte Java;
2. `README.md`;
3. programa de demonstração executável.

4 Estrutura Específica do Grupo

Estrutura: BST (árvore binária de busca) simples — sem balanceamento — indexada por timestamp.

A BST funciona como **índice ordenado por tempo**: cada nó armazena um par (`timestamp`, `valor`), com `timestamp` servindo de chave de ordenação. Isso permite responder consultas de **soma em faixa** percorrendo a árvore em ordem.

4.1 Requisitos da BST

- classe `NoBST` com atributos `timestamp` (`int`), `valor` (`double`), `esquerda` e `direita`;
- método `inserir(int timestamp, double valor)`;
- método `somarFaixa(int inicio, int fim)` que devolve a soma dos valores cujos timestamps estão no intervalo `[inicio, fim]` (inclusive em ambas as pontas).

Dica de implementação para `somarFaixa`: percorrer a BST em ordem e acumular apenas os nós dentro da faixa. Para grupos confortáveis com recursão, é possível **podar** ramos cuja faixa não interceptam — não é exigido, mas vale como bom exercício de raciocínio sobre BSTs.

4.2 Remoção

A remoção física na BST **não** é exigida. Quando o usuário chamar `delete(timestamp)`, basta remover do `HashMap`. A BST pode reter o nó ou ser regenerada ao reiniciar.

5 Funcionalidade Específica do Grupo

Implementar:

```
public double consultarSoma(int inicio, int fim);
```

O método deve devolver a soma das leituras cujos timestamps estão em `[inicio, fim]`, usando a BST.

A demonstração deve:

1. inserir **8 a 16 leituras** com timestamps fora de ordem (por exemplo, `100, 250, 50, 400, 175, ...`);
2. executar `consultarSoma` para a faixa completa;
3. executar `consultarSoma` para uma faixa parcial (por exemplo, `[100, 300]`);
4. atualizar uma leitura existente e mostrar que a nova soma reflete a atualização.

Exemplo de saída esperada:

```
Soma [0, 999]      = 187,45
Soma [100, 300]    = 96,30
Após atualizar t=175 para 30.0:
Soma [100, 300]    = 102,80
```

6 Roteiro Sugerido de 3 Horas

Etapas	Tempo	Atividade
1	45 min	Adaptar <code>AULA15/src</code> (núcleo comum)
2	1 h 15 min	Classe <code>NoBST</code> com <code>inserir</code> e <code>somarFaixa</code>
3	45 min	Integrar a BST à <code>Telemetria</code> (<code>put</code> insere na BST)
4	15 min	Demonstração com 8 a 16 leituras

7 Critérios de Avaliação

Critério	Peso
<code>put/get/delete</code> corretos	3,0
Persistência e reconstrução do estado	2,5
Uso correto da BST (<code>inserir</code> , <code>somarFaixa</code>)	2,0

Critério	Peso
Organização, clareza do código e README	0,5
Total	8,0

8 Sugestão de Divisão de Tarefas

Grupos com 2 alunos

- **Aluno 1:** API + integração da BST + demonstração.
- **Aluno 2:** log + reconstrução + README.

Grupos com 3 alunos

- **Aluno 1:** núcleo + interface.
- **Aluno 2:** persistência e reconstrução.
- **Aluno 3:** classe `NoBST` e `consultarSoma()`.

9 Arquivos do Projeto: Exemplo da Aula 15 vs. Trabalho do Grupo

O **Material Complementar da Aula 15 no Educ@** contém o projeto Java de referência (`AULA15/src/`). A tabela abaixo especifica, para cada arquivo do projeto do grupo, se ele deve ser **usado como está, adaptado a partir do exemplo** ou **criado do zero**.

Arquivo do projeto do grupo	Origem	Ação do grupo
<code>SGBD.java</code>	Exemplo da Aula 15	Usar como está. Interface do TAD do banco.
<code>TipoOperacao.java</code>	Exemplo da Aula 15	Usar como está. Enum com <code>PUT</code> e <code>DEL</code> .
<code>RegistroLog.java</code>	Exemplo da Aula 15	Usar como está. Codifica e decodifica linhas do log.
<code>LogAppendOnly.java</code>	Exemplo da Aula 15	Usar como está.
<code>Telemetria.java</code>	Adaptado de <code>BancoChaveValor.java</code>	Renomear a partir do <code>BancoChaveValor.java</code> . Manter o <code>HashMap</code> por timestamp e integrar a BST como índice ordenado.
<code>DemoGrupo07.java</code>	Adaptado de <code>DemoAula15.java</code>	Renomear a partir do <code>DemoAula15.java</code> . Inserir 8 a 16 leituras e chamar <code>consultarSoma()</code> em duas faixas.
<code>NoBST.java</code>	—	Criar do zero pelo grupo. Nó da BST por timestamp com <code>inserir</code> e <code>somarFaixa</code> (percurso em ordem somando os valores dentro da faixa).

Somente os arquivos marcados como **Criar do zero pelo grupo** exigem código novo. Os demais são reaproveitados do projeto da Aula 15, com ou sem renomeação e adaptação.

10 Forma de Entrega

- **arquivo compactado .zip** enviado pelo **Educ@**;
- envio até **08/06/2026, 23h59**;
- **não haverá apresentação presencial**: a avaliação será feita exclusivamente sobre o material entregue.

11 Observações Finais

- não é necessário interface gráfica nem SQL;
- a poda de ramos em **somarFaixa** é desejável mas **não** é exigida — um percurso em ordem simples com filtro já atende ao escopo;
- **não há ponto extra por sofisticação**.

Regra prática

Se faltar tempo:

1. **put/get/delete**;
2. persistência;
3. BST + **consultarSoma**.

Prof. Alysson Martins Bruno *Estrutura de Dados — UNITINS — 2026.1*